

UNIVERSITY OF KHARTOUM

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

AI Multi-Agent UAV Detection System
A Distributed Edge-Computing
Framework
for Real-Time Anti-UAV Detection and
Tracking

Project Report

Authors:

[Author Names]

Supervisor:

[Supervisor Name]

March 15, 2025

Contents

1	Introduction	3
1.1	Background of the Study	3
1.2	Problem Statement	4
1.3	Significance of the Project	4
1.4	Project Objectives	5
1.5	Scope and Limitations	5
1.6	Methodology Overview	6
1.7	Organization of the Report	6
2	Theoretical Background and Literature Review	7
2.1	Part I: Theoretical Background	7
2.1.1	Fundamentals of Object Detection	7
2.1.2	Detection Architectures: One-Step vs. Two-Step	8
2.1.3	The YOLO Family: Evolution and Key Innovations	8
2.1.4	YOLOv12: Architecture	9
2.1.5	YOLO26: Architecture and Innovations	9
2.1.6	ByteTrack: Multi-Object Tracking	9
2.1.7	Theoretical Framework: Distance and Trajectory Estimation	10
2.1.8	Thermal Imaging for UAV Detection	10
2.1.9	Edge Computing for Real-Time Inference	11
2.1.10	MQTT Protocol for IoT Communication	11
2.1.11	WebRTC for Live Video Streaming	11
2.2	Part II: Literature Review	11
2.2.1	Deep Learning for Object Detection	11
2.2.2	YOLO Family for Aerial Detection	12
2.2.3	Drone vs. Bird Classification	12
2.2.4	Comparative Analysis of Detection Architectures	12
2.2.5	Research Gap and Project Contribution	12
3	Methodology	14

3.1	System Architecture Overview	14
3.2	Hardware and Network Setup	15
3.3	Model Training Experiments	15
3.3.1	Experiment 1: YOLOv12 Medium — Single-Class Drone Detection	15
3.3.2	Experiment 2: YOLO26 Small — Drone vs. Bird Classification . .	17
3.3.3	Comparative Summary	22
3.4	Edge Device Implementation	22
3.4.1	Camera Acquisition	22
3.4.2	Inference Pipeline	22
3.4.3	MQTT Communication and Security	22
3.5	Main Device Stack	23
3.6	Frontend Command Center	23
3.7	Launcher GUIs	25
3.8	Testing	25

Chapter 1

Introduction

1.1 Background of the Study

In recent years, the field of computer vision has undergone a radical transformation, shifting from manually engineered systems to those powered by artificial intelligence. Within this domain, object detection serves as a fundamental building block for modern engineering applications, particularly in autonomous systems and national security [1]. The evolution of aircraft and unmanned aerial vehicle (UAV) detection has transitioned from traditional radar-based and optical systems toward deep learning-based frameworks capable of simultaneously solving localization and classification in a single forward pass.

Current technological trends emphasize the use of Deep Convolutional Neural Networks (CNNs) to bridge the “semantic gap” that previously hindered image understanding [2]. Real-world applications of these systems are now widespread, ranging from autonomous drone navigation and civil aviation safety to advanced anti-aircraft defense systems. The integration of high-speed algorithms like the You Only Look Once (YOLO) family has allowed for real-time monitoring of airspace, which is critical in scenarios where a fraction of a second can determine the success of a detection or intercept mission [6].

The proliferation of commercially available mini-drones has introduced a new class of security threats. These compact, highly maneuverable aerial devices are increasingly exploited for unauthorized surveillance, smuggling, and potential weaponization [7]. Unlike conventional aircraft, mini-UAVs present a minimal electromagnetic signature, making them difficult to detect with legacy radar systems. This motivates the integration of visual detection systems capable of operating in real time on low-power edge hardware.

1.2 Problem Statement

Despite significant advancements, detecting UAVs remains a formidable engineering challenge due to extreme scale variations, varying environmental lighting, and the “small object” problem. Traditional aircraft detection relies heavily on manual optical observation or expensive radar clusters, both of which have significant shortcomings. Optical methods are highly subjective and weather-dependent, while legacy radar systems often struggle with computational complexity and low-resolution output for smaller aerial targets.

Existing deep learning solutions often face a trade-off between accuracy and inference speed: two-step detectors are often too slow for active defense, while one-step detectors historically struggle with high-precision localization of distant aircraft [1]. Furthermore, most existing research treats detection and command-and-control as separate, loosely coupled pipelines, introducing latency that is unacceptable for real-time anti-UAV applications.

A particularly challenging sub-problem is the visual confusion between UAVs and birds, which share similar size, shape, and flight characteristics at distance. A robust detection system must reliably distinguish between these two classes to avoid false alarms in operational environments.

1.3 Significance of the Project

This project is significant for both its academic and practical contributions:

- **Academic Importance:** It explores the limits of NMS-free inference and architectural simplification in deep learning, providing a benchmark for the latest multi-task vision frameworks (YOLO26) on aerial target profiles, including the challenging drone-vs-bird discrimination task.
- **Practical Relevance:** It addresses the strategic need for indigenous air defense capabilities, specifically for regions requiring cost-effective alternatives to foreign technology. The system is designed to run on commodity laptop hardware connected via a standard WiFi network.
- **Systems Engineering Contribution:** The project implements a complete, production-ready distributed system integrating edge inference, secure MQTT communication, WebRTC live streaming, PTZ camera control, and a real-time web-based command center — all within a containerized, reproducible deployment.

1.4 Project Objectives

The general objective of this study is to design, implement, and evaluate a distributed AI multi-agent system for the real-time detection, tracking, and monitoring of aerial UAV targets. Specific objectives include:

1. To train and evaluate deep learning models (YOLOv12 and YOLO26) for UAV detection and drone-vs-bird classification.
2. To design a secure, low-latency communication architecture using MQTT over TLS for transmitting detection payloads from edge agents to a central aggregation service.
3. To implement a real-time WebRTC-based live video streaming pipeline from edge cameras to a web-based command center.
4. To develop a full-stack command-and-control interface providing device monitoring, geospatial mapping, PTZ camera control, and model hot-swapping.
5. To evaluate system robustness under different environmental conditions including thermal imaging, varying lighting, and network interruptions.

1.5 Scope and Limitations

This study covers the training and deployment of one-step deep learning detectors for real-time UAV detection on edge devices, integrated with a distributed command-and-control system. The research focuses on terrestrial edge-computing hardware (commodity laptops) connected via a local WiFi network.

Limitations include:

- Training datasets are limited to publicly available drone and bird imagery; domain-specific fine-tuning on local airspace data would further improve performance.
- Physical PTZ hardware integration is optional and driver-dependent.
- GPS-based geolocation relies on manual coordinate entry in the absence of dedicated GPS hardware on edge devices.
- Distance and trajectory estimation are implemented as theoretical extensions and have not been validated against ground-truth measurements.

1.6 Methodology Overview

The methodology begins with a theoretical study of deep learning-based object detection and the YOLO model family, followed by experimental training of two model variants. The system is then implemented as a distributed edge-computing architecture using Python, FastAPI, Node.js, and React/TypeScript, deployed via Docker Compose. Communication security is enforced via mutual TLS authentication. Performance is evaluated based on mAP50, mAP50-95, precision, recall, and training convergence curves.

1.7 Organization of the Report

This report is organized into three chapters. Chapter 2 provides the theoretical background and literature review. Chapter 3 details the system design, implementation, and experimental results.

Chapter 2

Theoretical Background and Literature Review

2.1 Part I: Theoretical Background

2.1.1 Fundamentals of Object Detection

Object detection is defined as a dual-mission computer vision task consisting of *classification* (identifying what an object is) and *localization* (identifying where the object is within an image). In deep learning, this is achieved through Convolutional Neural Networks (CNNs), which use hierarchical feature representation to progressively extract abstract features from raw pixel data [2].

Key concepts include:

- **Feature Maps:** Three-dimensional matrices representing pixel intensities transformed through convolutional layers to identify textures, edges, and eventually high-level semantic features such as aircraft wings or fuselages.
- **Receptive Field:** The specific portion of the input image that a particular CNN neuron responds to. Larger receptive fields capture global context; smaller ones capture fine-grained local detail.
- **Bounding Boxes:** Defined by coordinates (x, y, w, h) representing the center position and dimensions of a detected object's spatial extent.
- **Confidence Score:** A scalar $c \in [0, 1]$ representing the model's certainty that a bounding box contains an object of the predicted class.
- **Mean Average Precision (mAP):** The primary evaluation metric for object

detectors. mAP50 measures average precision at IoU threshold 0.5; mAP50-95 averages over IoU thresholds from 0.5 to 0.95 in steps of 0.05, providing a stricter measure of localization quality.

2.1.2 Detection Architectures: One-Step vs. Two-Step

Region Proposal-Based (Two-Step) Detectors

Systems such as R-CNN [4] and Faster R-CNN [5] operate in two stages: first generating candidate regions of interest via a Region Proposal Network, then classifying each region independently. These architectures achieve high accuracy but are computationally expensive, typically achieving only 5–7 FPS on standard hardware, making them unsuitable for real-time anti-UAV applications.

Regression-Based (One-Step) Detectors

One-step detectors such as YOLO [3] and SSD map pixels directly to bounding box coordinates and class probabilities in a single forward pass. This design achieves real-time inference speeds (30–200+ FPS depending on model size and hardware) at the cost of slightly reduced accuracy on small or densely packed objects. For anti-UAV applications where latency is critical, one-step detectors are the preferred architecture.

2.1.3 The YOLO Family: Evolution and Key Innovations

The YOLO family, introduced by Redmon et al. in 2016 [3], has undergone continuous architectural evolution. Table 2.1 summarizes the key milestones relevant to this work.

Table 2.1: Evolution of the YOLO Model Family

Model	Year	Key Innovation
YOLOv1	2016	First unified single-stage detector
YOLOv3	2018	Darknet-53 backbone, multi-scale feature maps
YOLOv5	2020	PyTorch, SiLU activation, anchor-free option
YOLOv8	2023	Decoupled head, C2f backbone, fully anchor-free
YOLOv10	2024	End-to-end NMS-free via dual-assignment training
YOLOv12	2025	Area attention module, R-ELAN blocks
YOLO26	2025	NMS-free, DFL-free, MuSGD, STAL, ProgLoss

2.1.4 YOLOv12: Architecture

YOLOv12 introduced an attention-centric architecture featuring an efficient area attention module that achieves global self-attention with low computational complexity, and Residual ELAN (R-ELAN) blocks for improved feature aggregation. This design achieves transformer-level accuracy at YOLO inference speeds, making it well-suited for fine-grained detection tasks such as drone identification. In this project, YOLOv12 Medium was used for the initial single-class drone detection experiment (Test 1).

2.1.5 YOLO26: Architecture and Innovations

YOLO26, released by Ultralytics in September 2025, represents the most significant architectural departure in the YOLO lineage [6]. Its key innovations are:

- **NMS-Free Inference:** Eliminates the Non-Maximum Suppression post-processing bottleneck through an end-to-end prediction strategy, reducing inference latency by up to 43% on CPU platforms.
- **Removal of Distribution Focal Loss (DFL):** Replaces the DFL regression head with a simplified variant, reducing model complexity without sacrificing accuracy.
- **MuSGD Optimizer:** A momentum-based SGD variant providing more stable convergence, particularly on small and imbalanced datasets typical of aerial surveillance scenarios.
- **Small-Target-Aware Label Assignment (STAL):** A specialized label assignment strategy that improves detection of small objects — critical for detecting mini-UAVs at distance.
- **ProgLoss (Progressive Loss):** A curriculum-style loss function that gradually increases task difficulty during training, improving generalization to challenging real-world conditions.

YOLO26 Small (YOLO26s) was used in Test 2 for the two-class drone-vs-bird discrimination experiment.

2.1.6 ByteTrack: Multi-Object Tracking

Detection alone is insufficient for anti-UAV applications; persistent tracking of individual targets across frames is required to estimate trajectories and predict future positions. ByteTrack [8] is a high-performance multi-object tracking algorithm that associates detections across frames using the BYTE association strategy. Unlike methods that discard low-confidence detections, ByteTrack retains them as secondary candidates, significantly

improving tracking continuity for partially occluded or distant targets. Each tracked object is assigned a persistent `track_id`, enabling trajectory analysis.

2.1.7 Theoretical Framework: Distance and Trajectory Estimation

While not validated experimentally in this work, the theoretical framework for distance and trajectory estimation is presented as a basis for future extension.

Distance Estimation via Pinhole Camera Model

Using the pinhole camera model, the estimated distance to a target can be derived from the apparent size of its bounding box:

$$d = \frac{S_{\text{ref}} \cdot f}{w_{\text{bbox}}} \quad (2.1)$$

where d is the estimated distance in meters, S_{ref} is the known reference size of the target (e.g., 0.5 m for a typical mini-UAV), f is the focal length in pixels (derived from the camera's field of view angle θ and frame width W as $f = W/(2 \tan(\theta/2))$), and w_{bbox} is the detected bounding box width in pixels. This model assumes a frontal view and a known target size, both of which are reasonable approximations for UAV detection at moderate distances.

Trajectory Estimation via Sliding Window

Trajectory can be estimated as the mean frame-to-frame displacement of a tracked object's bounding box center over a sliding window of N frames:

$$\vec{v} = \frac{1}{N-1} \sum_{i=1}^{N-1} (\vec{c}_i - \vec{c}_{i-1}) \quad (2.2)$$

where $\vec{c}_i = (x_i + w_i/2, y_i + h_i/2)$ is the bounding box center at frame i . The resulting trajectory vector $\vec{v} = (\Delta x, \Delta y)$ represents the average pixel displacement per frame, which can be converted to angular velocity given known camera parameters. This information is valuable for predicting future target positions and cueing PTZ cameras.

2.1.8 Thermal Imaging for UAV Detection

Thermal (infrared) cameras capture radiation emitted by objects rather than reflected visible light, enabling detection in low-light conditions, fog, smoke, and at night [9].

Mini-UAVs are identifiable by their heat signatures from motors and electronics. However, thermal imagery introduces unique challenges including low contrast, thermal crossover effects, and domain gap relative to RGB-trained models.

To address these challenges, the system implements CLAHE (Contrast Limited Adaptive Histogram Equalization) preprocessing followed by the INFERNO colormap to convert thermal grayscale frames into a three-channel representation compatible with standard YOLO models.

2.1.9 Edge Computing for Real-Time Inference

Edge computing refers to the paradigm of processing data at or near the source of generation [10]. For anti-UAV detection, edge computing offers reduced latency, bandwidth efficiency (only structured JSON payloads transmitted), resilience during network interruptions, and privacy (raw video never leaves the edge device unless explicitly requested).

2.1.10 MQTT Protocol for IoT Communication

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained IoT devices [12]. Key features include three QoS levels, Last Will and Testament (LWT) for offline detection, retained messages for status persistence, and TLS mutual authentication for secure device identity verification.

2.1.11 WebRTC for Live Video Streaming

WebRTC (Web Real-Time Communication) is an open standard for peer-to-peer audio, video, and data communication [13]. It provides sub-second latency, mandatory DTLS-SRTP encryption, ICE-based NAT traversal, and adaptive bitrate control — all essential properties for a real-time surveillance streaming pipeline.

2.2 Part II: Literature Review

2.2.1 Deep Learning for Object Detection

Zhao et al. (2019) [1] provided a comprehensive survey of object detection methods, establishing the foundational taxonomy of two-stage vs. one-stage detectors. Girshick et al. (2014) [4] introduced R-CNN, while Ren et al. (2015) [5] unified region proposal and classification into Faster R-CNN, achieving $\sim 200\text{ms}$ per image — still insufficient for real-time anti-UAV applications.

2.2.2 YOLO Family for Aerial Detection

Redmon et al. (2016) [3] introduced the original YOLO architecture, achieving 45 FPS while maintaining competitive accuracy. Reis et al. (2023) [7] demonstrated YOLOv8 for real-time flying object detection, achieving mAP50 of 99.1% on a refined drone dataset after transfer learning from a 40-class flying object dataset. Sapkota and Karkee (2025) [6] demonstrated YOLO26 achieving a 43% CPU speedup over YOLOv8 through NMS and DFL removal, with competitive mAP on NVIDIA Jetson edge hardware.

2.2.3 Drone vs. Bird Classification

The visual similarity between drones and birds at distance is a well-documented challenge in aerial surveillance. Both share similar apparent sizes, aspect ratios, and flight trajectories when observed from ground-level cameras. Delleji et al. (2024) [9] highlighted this challenge in the thermal domain, noting that mini-drones and birds share thermal characteristics, making accurate identification non-trivial. The “Drone vs Bird” dataset used in Test 2 of this project directly addresses this challenge by providing labeled examples of both classes under real-world conditions.

2.2.4 Comparative Analysis of Detection Architectures

Table 2.2: Comparison of Detection Architectures for Aerial Surveillance

Architecture	Type	Strengths	Weaknesses	FPS
Faster R-CNN	Two-Step	High accuracy	Slow (~ 7 FPS)	7
YOLOv5	One-Step	Modular, PyTorch	NMS bottleneck	140
YOLOv8	One-Step	Anchor-free	DFL overhead	160
YOLOv12	One-Step	Area attention	Higher complexity	120
YOLO26	One-Step	NMS-free, edge-opt.	Newest, less validated	200+

2.2.5 Research Gap and Project Contribution

While the transition to deep learning has solved many issues related to manual feature extraction, a significant gap remains in the native integration of detection, tracking, and command-and-control into a unified, deployable system for low-power edge devices. Most existing research treats detection and control as separate pipelines. Furthermore, the drone-vs-bird discrimination problem remains underexplored in the context of real-time edge deployment.

This project addresses these gaps by: (1) experimentally evaluating YOLOv12 and YOLO26

on drone detection and drone-vs-bird classification tasks; (2) implementing a complete distributed anti-UAV detection system integrating YOLO26 inference, ByteTrack tracking, MQTT-based secure telemetry, WebRTC live streaming, and a full-stack React command center; all deployable on commodity hardware via Docker Compose.

Chapter 3

Methodology

3.1 System Architecture Overview

The proposed system follows a distributed edge-computing architecture comprising three tiers: edge agents performing local inference, a main device running the aggregation and command-and-control stack, and a web-based frontend accessible from any browser. The architecture is illustrated conceptually below:

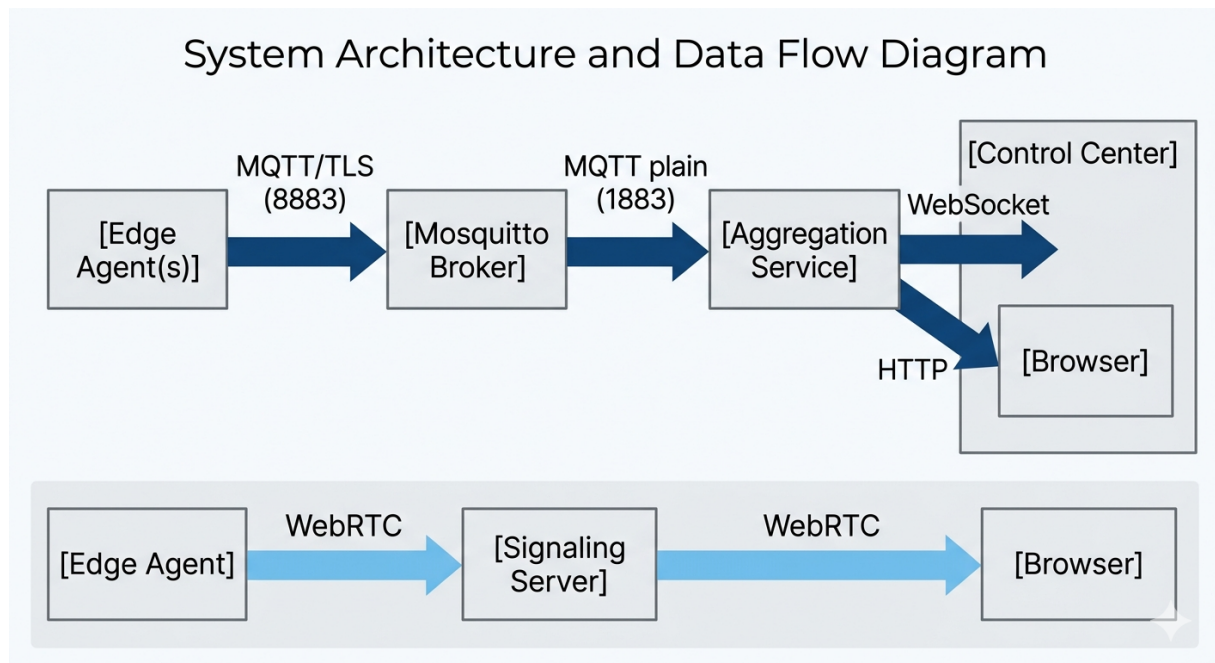


Figure 3.1: High-level distributed system architecture

3.2 Hardware and Network Setup

The system was implemented and tested on two commodity laptops running CachyOS (Arch Linux), connected via a shared WiFi hotspot. An Android smartphone running the IP Webcam application provided an MJPEG video stream over HTTP. The network topology assigns addresses in the 192.168.x.x/24 subnet, with the main device hosting the Docker stack and the edge device running the Python inference pipeline.

3.3 Model Training Experiments

Two training experiments were conducted to evaluate different model architectures and task formulations for UAV detection.

3.3.1 Experiment 1: YOLOv12 Medium — Single-Class Drone Detection

Configuration

The first experiment trained a YOLOv12 Medium model on a single-class drone detection dataset. Training was conducted for 100 epochs with a batch size of 8 at 640×640 resolution using the SGD optimizer with cosine annealing learning rate schedule.

Table 3.1: Experiment 1 Training Configuration

Parameter	Value
Model	YOLOv12 Medium
Task	Object Detection (single class: drone)
Image size	640×640
Epochs	100
Batch size	8
Optimizer	SGD with cosine annealing
Classes	1 (drone)

Results

Table 3.2: Experiment 1 — Best Validation Metrics (Epoch 73)

Metric	Value
mAP50	0.9159
mAP50-95	0.4926
Precision	0.9879
Recall	0.7778
Final train box loss	0.8591
Final val box loss	1.7353
Total training time	~44 minutes

The model achieved a peak mAP50 of **91.59%** at epoch 73, with near-perfect precision of 98.79%. The recall of 77.78% indicates that while the model is highly confident in its positive detections, it misses approximately one in five drone instances — likely small or partially occluded targets. The gap between mAP50 (0.916) and mAP50-95 (0.493) suggests that while the model correctly identifies drone locations at a relaxed IoU threshold, tighter localization accuracy requires further improvement.

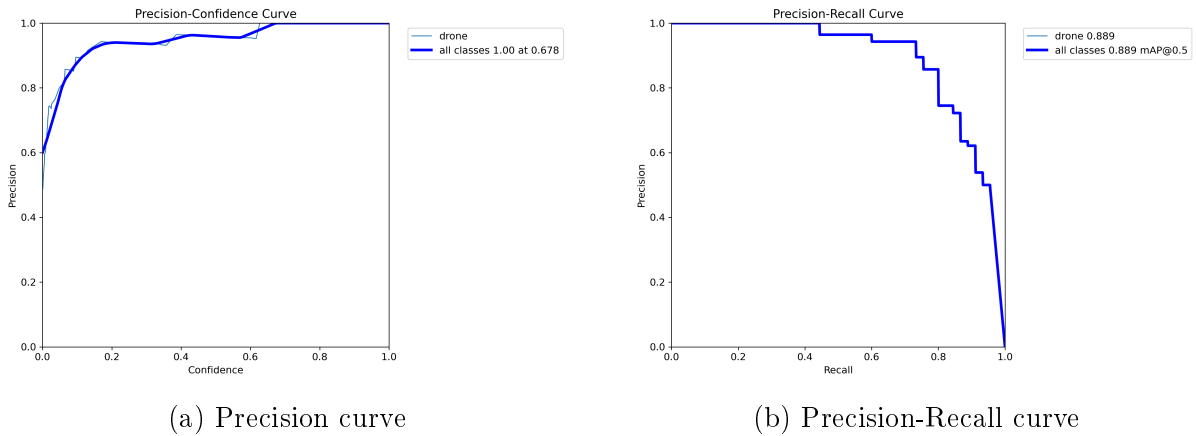


Figure 3.2: Experiment 1 — YOLOv12 Medium detection performance curves

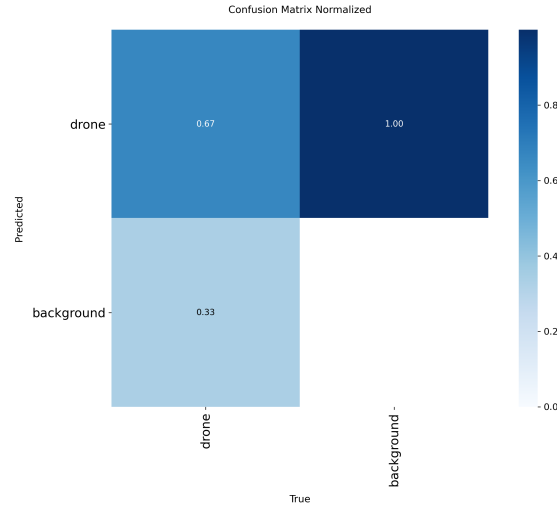


Figure 3.3: Experiment 1 — Normalized confusion matrix

3.3.2 Experiment 2: YOLO26 Small — Drone vs. Bird Classification

Configuration

The second experiment trained a YOLO26 Small model on the “Drone vs Bird v3” dataset, a two-class detection task requiring the model to distinguish between drones and birds — a significantly harder problem due to visual similarity between the two classes. Training was conducted for 160 epochs with a batch size of 16 at 896×896 resolution using the AdamW optimizer with cosine learning rate scheduling.

Table 3.3: Experiment 2 Training Configuration

Parameter	Value
Model	YOLO26 Small (yolo26s.pt)
Task	Object Detection (2 classes: drone, bird)
Dataset	Drone vs Bird v3 (YOLO format)
Image size	896×896
Epochs	160
Batch size	16
Optimizer	AdamW
Learning rate	0.002 (cosine decay to $0.01 \times$)
Warmup epochs	5
Augmentation	Mosaic (0.6), MixUp (0.1), RandAugment
Hardware	NVIDIA T4 GPU (Google Colab)

Dataset

The dataset contains labeled images of drones and birds captured under diverse real-world conditions including varying distances, lighting conditions, backgrounds, and flight orientations. Figure 3.4 shows the class distribution and bounding box statistics of the training set.

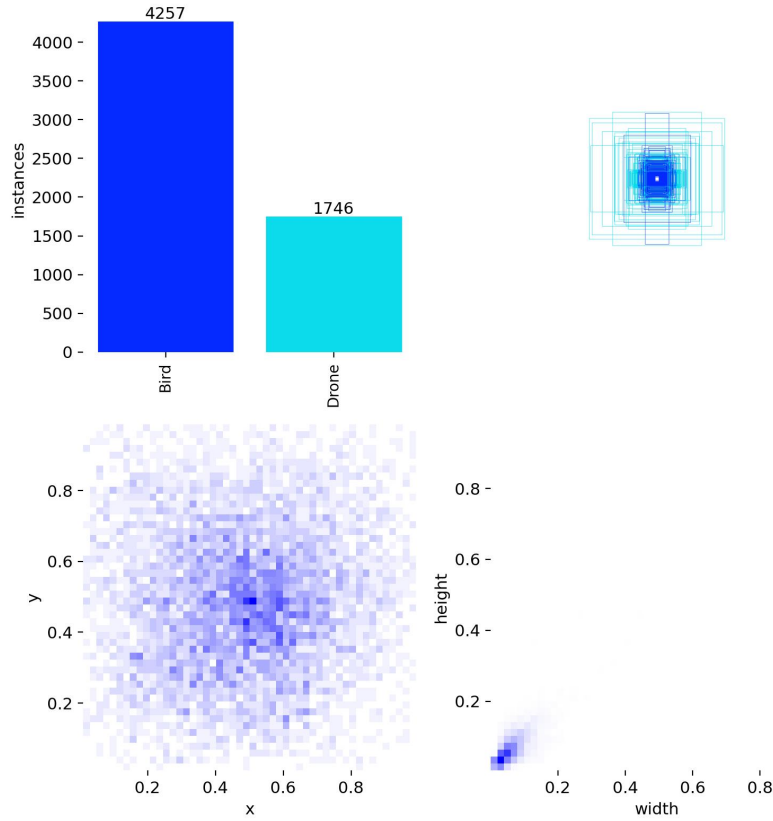


Figure 3.4: Experiment 2 — Dataset label distribution and bounding box statistics

Results

Table 3.4: Experiment 2 — Best Validation Metrics (Epoch 145)

Metric	Value
mAP50	0.8547
mAP50-95	0.4260
Precision	0.8341
Recall	0.8093
Final train box loss	1.5193
Final val box loss	1.8039
Total training time	~3.91 hours

The YOLO26s model achieved a peak mAP50 of **85.47%** on the two-class drone-vs-bird task at epoch 145. Compared to Experiment 1, the more balanced precision (83.41%) and recall (80.93%) indicate that the model generalizes better across both classes, avoiding the high-precision/low-recall trade-off observed in the single-class experiment. The training converged smoothly over 160 epochs, with the loss curves showing consistent improvement throughout.

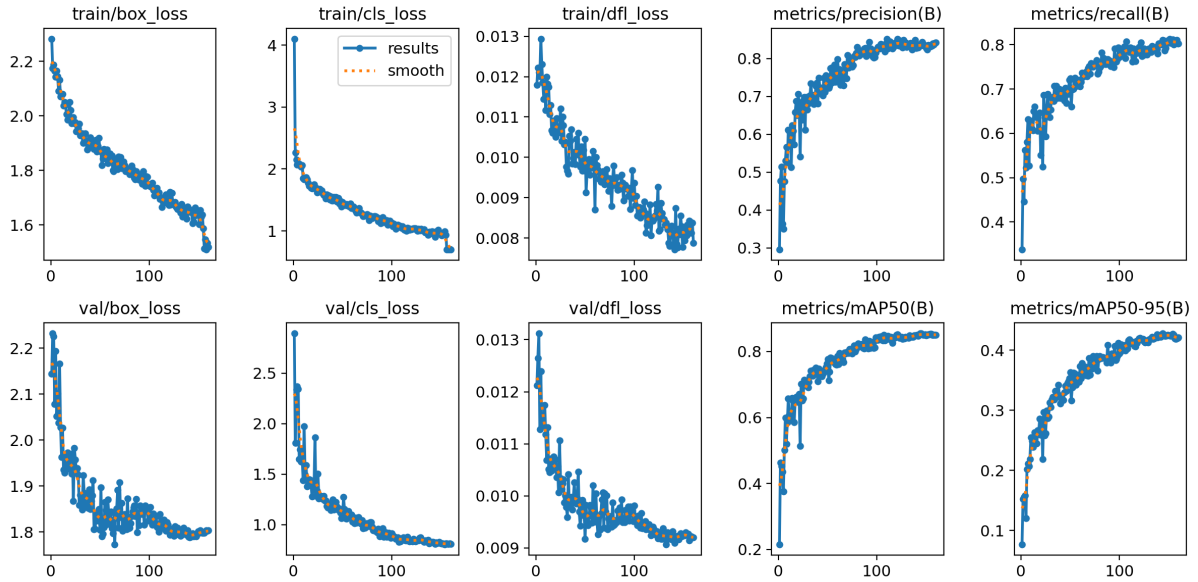


Figure 3.5: Experiment 2 — Training and validation metrics over 160 epochs

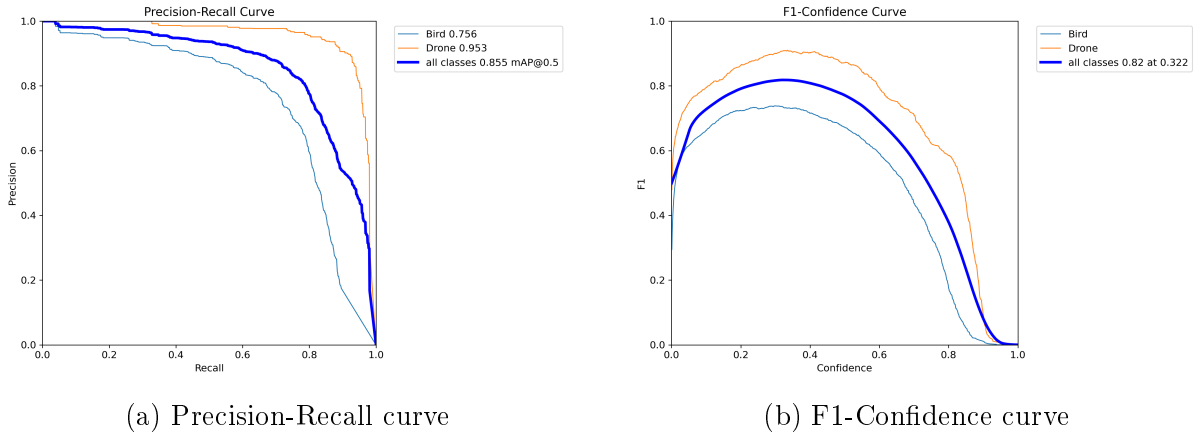


Figure 3.6: Experiment 2 — YOLO26s detection performance curves

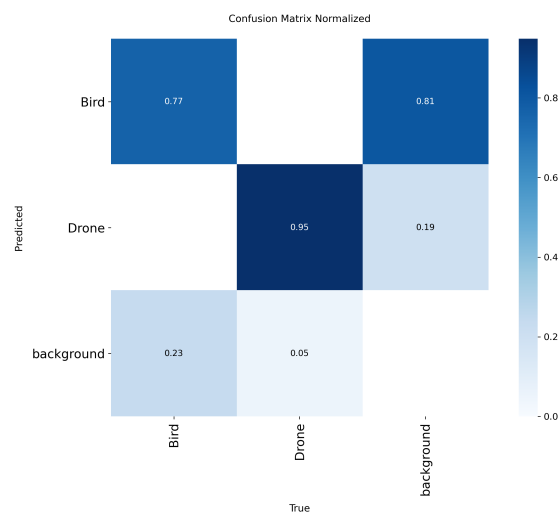


Figure 3.7: Experiment 2 — Normalized confusion matrix (drone vs. bird)

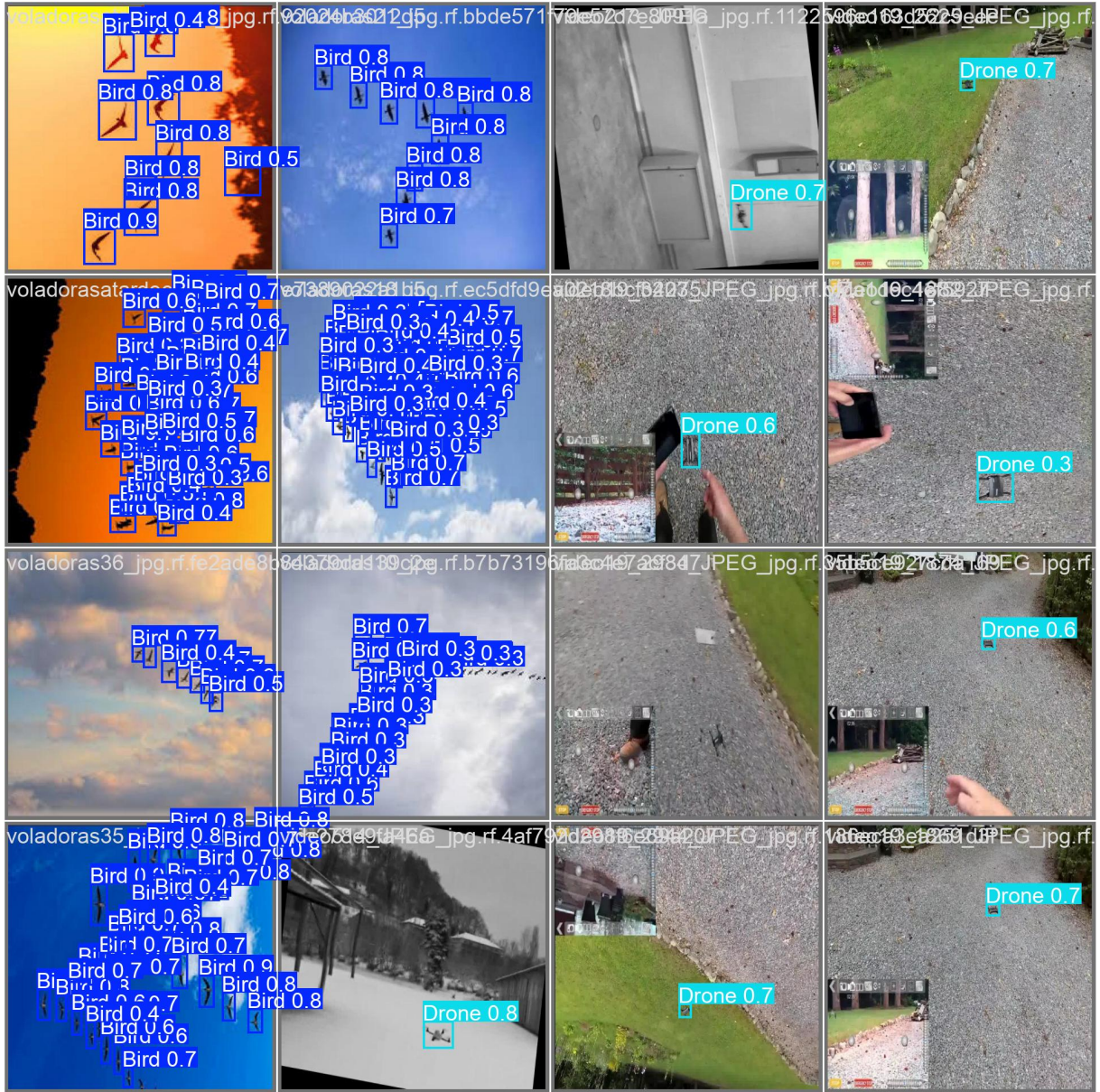


Figure 3.8: Experiment 2 — Sample validation predictions with bounding boxes

Discussion

The two experiments reveal complementary strengths. YOLOv12 Medium achieves higher mAP50 (91.6% vs 85.5%) on the simpler single-class task, benefiting from the focused training signal. YOLO26s, despite the harder two-class discrimination task, achieves more balanced precision-recall performance and demonstrates the effectiveness of YOLO26’s STAL and ProgLoss innovations for handling the visual ambiguity between drones and birds.

The drone-vs-bird confusion matrix (Figure 3.7) reveals the specific failure modes: the model occasionally misclassifies birds as drones at small apparent sizes, which is the expected failure mode given the visual similarity at distance. This suggests that incorpo-

rating temporal information (trajectory patterns) alongside spatial features could further improve discrimination accuracy — a direction for future work.

3.3.3 Comparative Summary

Table 3.5: Comparison of Training Experiments

Exp.	Model	Classes	mAP50	mAP50-95	Precision	Recall
1	YOLOv12-M	1 (drone)	0.916	0.493	0.988	0.778
2	YOLO26s	2 (drone+bird)	0.855	0.426	0.834	0.809

3.4 Edge Device Implementation

3.4.1 Camera Acquisition

The `CameraSource` module provides unified frame acquisition from USB cameras (`/dev/videoN`), RTSP streams, and HTTP MJPEG streams. HTTP sources use a custom JPEG boundary parser (scanning for `0xFF 0xD8` / `0xFF 0xD9` markers) since `opencv-python-headless` lacks FFMPEG support for HTTP URLs. Frames are pushed into a thread-safe queue at the configured target FPS, with automatic reconnection on disconnection.

3.4.2 Inference Pipeline

The inference pipeline processes frames through the following stages: (1) frame de-queue from camera buffer; (2) optional thermal preprocessing (CLAHE + INFERNO colormap); (3) YOLO inference with ByteTrack tracking via `model.track()`; (4) payload construction (device ID, timestamp, frame ID, detections); (5) MQTT publication to `uav/tracking/{device_id}`.

The `ModelManager` supports runtime model hot-swapping: inference is paused, the new model loaded in a background thread, then atomically swapped without restarting the edge process.

3.4.3 MQTT Communication and Security

All external MQTT communication uses mutual TLS authentication with X.509 certificates generated by the `certs/gen_certs.sh` script. The broker maintains two listeners: port 8883 (TLS, for edge agents) and port 1883 (plain, internal Docker network only). Tracking payloads use QoS 0 for minimal latency; status messages use QoS 1 with the

retained flag. Last Will and Testament messages enable automatic offline detection on unexpected disconnection.

3.5 Main Device Stack

The main device runs a Docker Compose stack comprising five services:

- **Mosquitto:** MQTT broker with dual listeners (TLS external / plain internal).
- **Aggregation Service:** FastAPI application subscribing to all UAV MQTT topics, maintaining a thread-safe device registry, and exposing REST and WebSocket APIs.
- **Control Center:** FastAPI application serving the React frontend and proxying API requests to the aggregation service.
- **Signaling Server:** Node.js WebSocket server implementing room-based WebRTC signaling (SDP offer/answer and ICE candidate relay).
- **Frontend Builder:** Vite/React build container that compiles the TypeScript frontend and copies the output to a shared Docker volume.

3.6 Frontend Command Center

The React/TypeScript frontend provides four views:

- **Dashboard:** Tabular device status with inline model switching.
- **Map:** Leaflet.js interactive map with OSM tiles, device markers (color-coded by status), detection alert markers, and an unlocated devices panel.
- **Live Feeds:** 2×2 WebRTC video grid with real-time bounding box overlays and compact PTZ controls.
- **PTZ Control:** Joystick-based pan/tilt control with zoom buttons and absolute position inputs.

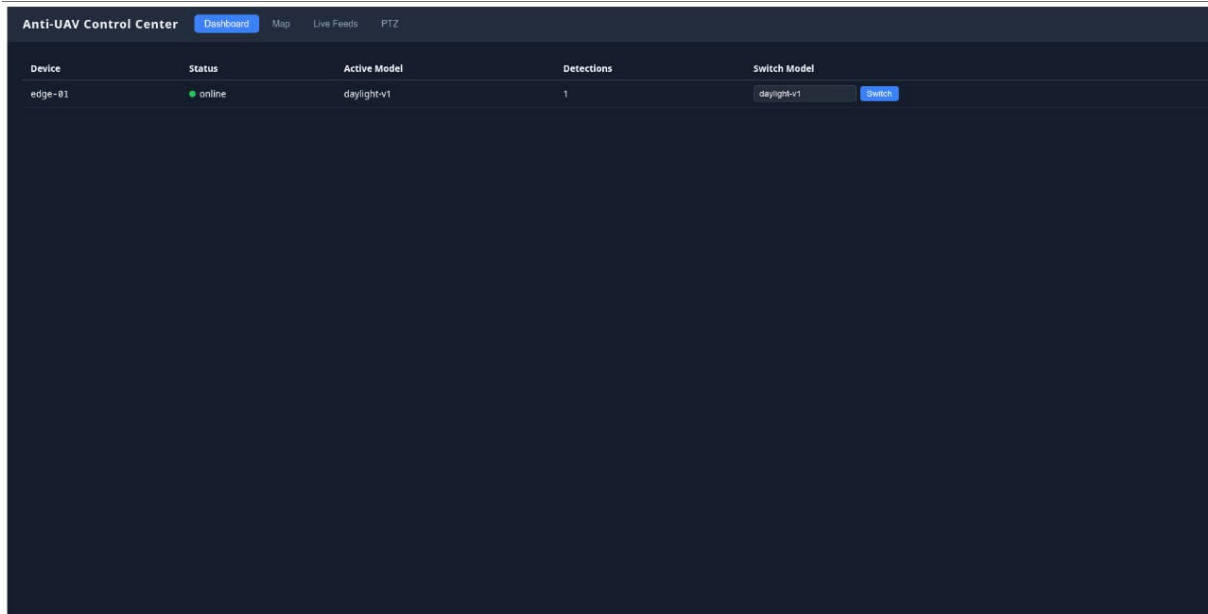


Figure 3.9: Anti-UAV Control Center — main dashboard view

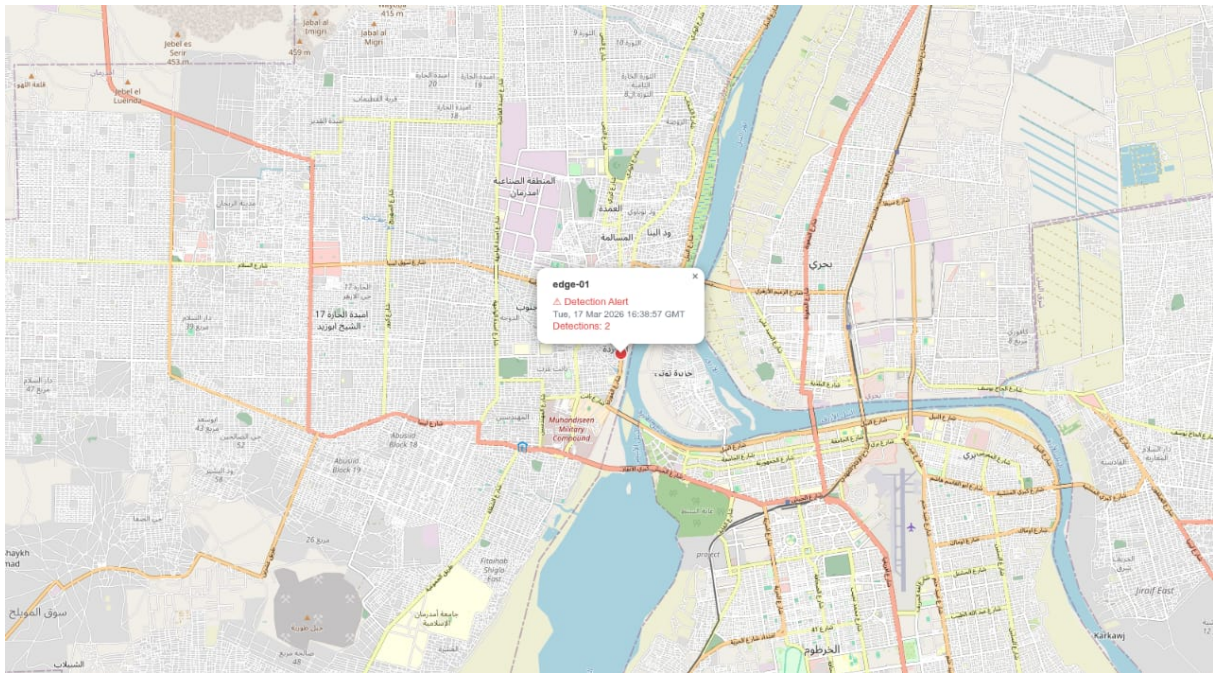


Figure 3.10: Map view — geospatial device tracking with OpenStreetMap tiles

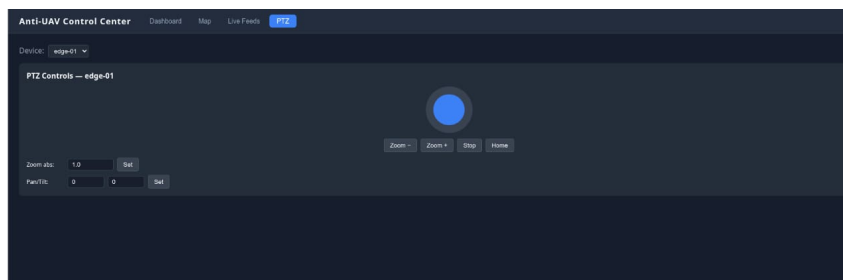


Figure 3.11: PTZ control panel — joystick pan/tilt and zoom controls

3.7 Launcher GUIs

Two Tkinter-based GUI launchers simplify system operation: `launcher_main.py` configures and starts the Docker stack on the main device; `launcher_edge.py` configures camera, MQTT, TLS certificates, and model paths on the edge device, then launches the inference process.

3.8 Testing

The system includes 138 passing unit and integration tests covering all components, implemented with `pytest`, `pytest-asyncio`, and `hypothesis` for property-based testing.

Bibliography

- [1] Z. Zhao, P. Zheng, S. Xu, and X. Wu, “Object Detection with Deep Learning: A Review,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 11, pp. 3212–3232, 2019.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [4] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *Proc. IEEE CVPR*, 2014, pp. 580–587.
- [5] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.
- [6] R. Sapkota and M. Karkee, “YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection,” *arXiv:2509.25164v4*, 2026.
- [7] D. Reis, J. Hong, J. Kupec, and A. Daoudi, “Real-Time Flying Object Detection with YOLOv8,” *arXiv:2305.09972v2*, 2024.
- [8] Y. Zhang *et al.*, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Proc. ECCV*, 2022, pp. 1–21.
- [9] T. Delleji, F. Slimeni, and A. Siala, “Good Data Beats More Data: Building a Thermal Air-Image Dataset for Ground-to-Air Surveillance,” in *Transfer Learning – Unlocking the Power of Pretrained Models*, IntechOpen, 2024.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

- [11] M. Stacker *et al.*, “Deployment of Deep Neural Networks for Object Detection on Edge Devices,” in *Proc. IEEE/CVF ICCV Workshops*, 2021.
- [12] OASIS Standard, “MQTT Version 5.0,” 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [13] W3C and IETF, “WebRTC 1.0: Real-Time Communication Between Browsers,” W3C Recommendation, 2021. [Online]. Available: <https://www.w3.org/TR/webrtc/>
- [14] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.